

web

Wednesday, 1 April 2026 16:59

11:15 58

←

<https://www.linkedin.com/in/masilvav/>

## Claude Code Leak

- What was actually leaked -

Diagram illustrating the Claude Code architecture and what was leaked:

- Claude LLM Core (Model Weights) - NOT Leaked**
- Claude Code CLI /Agent Harness (What was leaked)**
- Layered Memory System (3 layers)**
- Agent Loop**
- Tools (Git, Bash)**
- Feature Flags**
- KAIROS mode**
- Anthropic**

<https://www.linkedin.com/in/masilvav/>

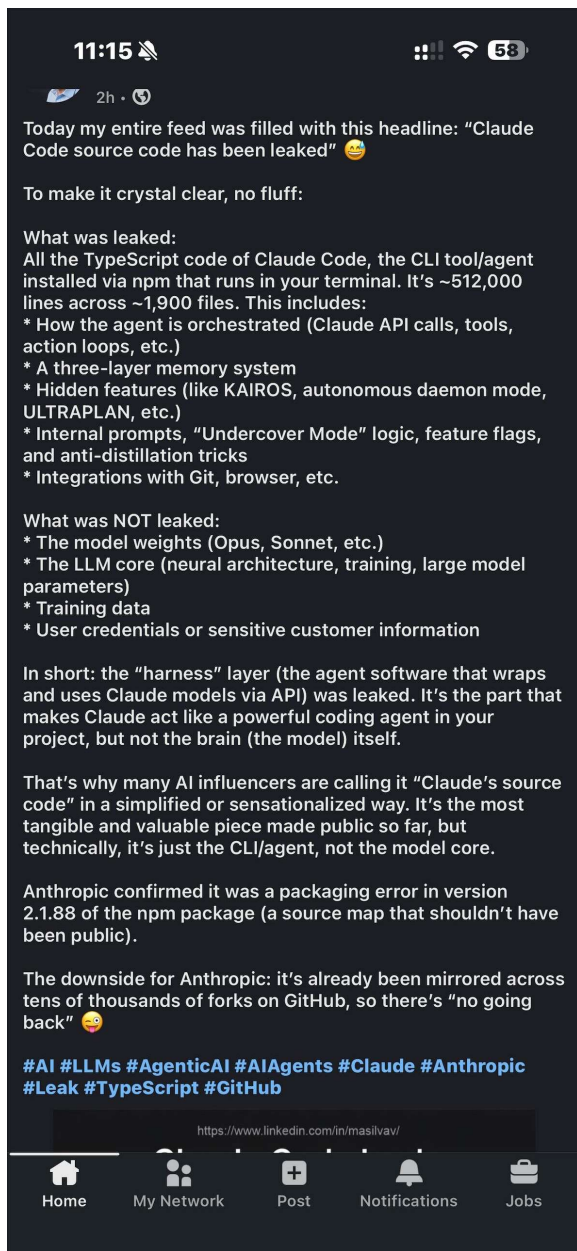
| ✓ Leaked           | ✗ NOT Leaked    |
|--------------------|-----------------|
| ✓ Agent Code       | ✗ Model Weights |
| ✓ Internal Prompts | ✗ Training Data |
| ✓ Memory           | ✗ Credentials   |
| ✗ Undercover Mode  |                 |

Leak via Source Map • 512,000 lines of TypeScript • ~1,900 files

**Mariano Silva** [Follow](#)

Data & AI Leader | Building Scalable Data...

Today my entire feed was filled with this headline:  
"Claude Code source code has been leaked"... more



# 我花了一晚上细品刚泄露的 Claude Code 源码，终于搞懂了它为什么这么强

全文5295字 | 阅读需17分钟

Claude Code 的源码最近泄露了。拿到之后我当晚就没睡，从头到尾读了一遍。说实话，读完之后我对 Anthropic 工程团队的敬佩又上了一个台阶。这套系统的设计哲学，跟我在硅谷这些年见过的绝大多数 agent 框架都不一样——它不是在炫技，而是在解决真实问题。

今天这篇文章，我会把 Claude Code 的 agent flow 从头到尾拆给大家看。不讲概念，只讲代码。

## 先说结论

市面上 90% 的 AI agent 框架都在纠结一个问题：怎么设计一个漂亮的 DAG（有向无环图）来编排工具

rednote ID: 10819659153

调用。

Claude Code 的回答是：不需要 DAG。

它的核心就是一个 `while(true)` 循环。没有状态机，没有图编排引擎，没有 workflow DSL。就是一个朴素到不能再朴素的循环：压缩上下文 → 调 API → 流式处理响应 → 执行工具 → 把结果塞回消息列表 → `continue`。

但这个"朴素"的循环里，藏着我见过的最精密的工程设计。

## 1. 入口：从用户敲下回车开始

用户在终端输入一段话，按下回车。系统从 CLI 入口进入 `QueryEngine.submitMessage()`，再进入 `processUserInput()` 做斜杠命令展开，最后进入核心的 `query()` 函数——整个 agent loop 的心脏。

`QueryEngine` 是会话的"管家"，干三件事：

第一，持久化消息——在调 API 之前就写入 transcript。为什么？因为如果 API 调用中途崩了，下次可以断点续传。这个细节很小，但体现了工程成

熟度。

第二，文件历史快照——每个 turn 开始时拍一次快照，支持 undo。你让 Claude Code 改了文件不满意，它能帮你回退，靠的就是这个。

第三，成本追踪——跨 turn 累计 token 用量和美元成本。

真正的核心逻辑，全部在 query() 里。

## 2. 主循环：Agent 的心脏

打开 query.ts，你会看到一个 queryLoop 函数。它是一个 async generator（async function\*），整个 Claude Code 的灵魂就在这里。

主循环每轮做七件事：① 解构可变状态 → ② 发出 stream\_request\_start 信号 → ③ 四层上下文压缩（Snip → Microcompact → Context Collapse → AutoCompact）→ ④ 流式调用 Claude API → ⑤ 流式解析响应（text/thinking yield 给 UI，tool\_use 收集起来）→ ⑥ 工具执行（并发/串行分批）→ ⑦ 判断 needsFollowUp，决定 continue

rednote ID: 10819659153

还是 return。

状态管理：一个被低估的设计决策

每次循环迭代共享一个 State 对象，里面有 messages 、 toolUseContext 、 autoCompactTracking 等字段。但最值得注意的是 transition 字段——它记录了上一次循环为什么 continue 。是因为工具调用需要继续？是因为 prompt 太长触发了压缩？还是因为 max\_output\_tokens 需要恢复？

这个字段的存在，让循环体内的恢复逻辑可以判断"我们是正常迭代，还是从某个错误恢复回来的"。比如 context collapse 排空后重试仍然 413，就不会再排第二次——因为 transition.reason === 'collapse\_drain\_retry'。

还有一个精妙的设计：每次 continue 时，整个 state 被一次性替换，不是逐字段赋值。为什么？因为 TypeScript 的类型系统能确保每个 continue site 不遗漏任何字段。代码里有 7 个不同的 continue site（工具调用继续、max\_tokens 恢复、reactive compact 重试、collapse drain 重



rednote ID: 10819659153

试等)，任何一个漏了字段，编译器就报错。

这是我见过的对 TypeScript 类型系统最优雅利用之一。

### 3. 流式处理：不是你以为的那种 streaming

大多数人理解的 streaming 是：API 返回一个字一个字地显示在屏幕上。

Claude Code 的 streaming 要复杂得多。它在流式接收 API 响应的同时，就开始执行工具了。

StreamingToolExecutor 的工作方式是：当一个 tool\_use block 完整到达时（不用等整个 API 响应结束），立刻开始执行。如果模型同时调了 3 个 Read 工具，这三个读操作是并行的。

这里有个微妙的工程问题：如果一个 Bash 工具执行出错了，正在并行执行的其他工具怎么办？

答案是 sibling abort controller——一个工具报错，会通过 abort signal 通知所有兄弟工具停止。然后为这些被 abort 的工具生成合成的 tool\_result

rednote ID: 10819659153

(告诉模型"这个工具被取消了")，确保 API 的 `tool_use / tool_result` 配对不会断裂。

## 4. 工具编排：为什么不用 DAG

这是我最想聊的部分。

`toolOrchestration.ts` 里有一个 `partitionToolCalls` 函数，它的规则只有两条：

第一，连续的并发安全工具（`Read`、`Grep`、`Glob` 等只读操作）合并为一个并发批次，最多 10 个并发。

第二，非并发安全工具（`Bash`、`Edit`、`Write` 等有副作用的操作）独占一个串行批次。

举个例子：输入 `[Read, Read, Grep, Bash, Read, Read]`，会被分成三批——`[Read, Read, Grep]` 并发执行 → `[Bash]` 串行执行 → `[Read, Read]` 并发执行。

没有 DAG，没有依赖分析，没有拓扑排序。就是一个简单的贪心分区算法。



rednote ID: 10819659153



为什么这就够了？因为依赖关系是模型决定的，不是框架决定的。模型在一次响应中输出的多个 `tool_use`，本身就隐含了“这些可以并行”的语义。框架只需要尊重一个不变量：有副作用的操作不能并发。这比那些需要用户手动画 DAG 的框架高明得多。

延迟应用上下文修改器

并发批次里还有一个精妙的设计。每个工具执行完可能需要修改共享的 `ToolUseContext`（比如更新文件缓存）。但并发执行时，多个工具同时修改同一个对象会产生竞态条件。

解决方案：先收集，后应用。并发执行所有工具时，修改器只收集不执行。批次结束后，按工具声明顺序依次应用修改器。并发执行，但确定性地应用副作用——这是并发编程的经典模式，在 agent 框架里很少见到有人做对。

## 5. 权限系统：安全不是 checkbox

Claude Code 的权限系统分四个等级：default（每个敏感操作弹框确认）、auto（安全分类器自动

rednote ID: 10819659153

审批)、plan (先出计划再执行)、bypass (跳过所有检查)。

auto 模式最有意思。它会用一个独立的分类器在 2 秒内判断这个工具调用是否安全。如果 2 秒内没结果, 回退到弹框确认。不会为了安全牺牲响应速度, 也不会为了速度跳过安全检查。

权限决策链: 先做静态规则匹配 (alwaysAllow / alwaysDeny / alwaysAsk), 然后 auto 模式走推测性安全分类 (2 秒超时), 最后交互模式弹出确认对话框让用户选择 Allow / Deny / Always Allow。

## 6. 子 Agent: 递归而非编排

当 Claude Code 需要并行处理多个子任务时, 它会调用 Agent 工具。这是整个系统最强大也最优雅的部分。

子 agent 不是新进程, 不是微服务, 不是独立的 workflow。它就是递归调用了同一个 query() 函数。

这意味着三件事:

第一，子 agent 自动继承所有能力——四层压缩、错误恢复、流式工具执行、权限检查，全部自动获得。不需要为子 agent 单独实现任何基础设施。

第二，无限嵌套——子 agent 里可以再生成子 agent。理论上可以无限递归，实际上受 token 预算限制。

第三，零额外概念——不需要学习什么 "crew"、"swarm"、"graph" 之类的新抽象。子 agent 就是一个受限版的主 agent。

### 隔离设计

子 agent 的隔离是选择性的。文件缓存是克隆的，父子互不污染；AbortController 是独立的，子 agent 被 abort 不影响父 agent；工具集可以受限（Explore agent 没有 Edit 权限）。但 AppState 是共享的——因为权限变更（用户点了 "Always Allow"）必须全局生效。

### 后台 Agent 和 Worktree 隔离

run\_in\_background: true 时，子 agent 在后台运行，父 agent 不等待它——这实现了真正的异步多

agent 并发。

isolation: "worktree" 时，子 agent 在一个临时的 git worktree 里工作——一个完整的仓库副本。改完了如果有变更，返回 worktree 路径和分支名；没变更，自动清理。这解决了一个真实痛点：你让 agent 尝试一个大胆的重构方案，如果搞砸了，主分支毫发无损。

## 7. 上下文窗口管理：真正的工程深水区

这是整篇文章的重头戏。Claude Code 能支持长达数小时的编程会话，靠的就是这套四层压缩系统。

第一层：Snip

最轻量。移除整个低价值 turn。比如一个工具调用返回了空结果，整个 turn 直接删掉。成本：零 API 调用，纯本地操作。

第二层：Microcompact

缓存感知的内联精简。对 tool\_result 的内容进行压缩，但保持消息结构不变。这一层跟 Anthropic 的 prompt caching 机制深度集成——它知道哪些消息

rednote ID: 10819659153

在缓存里、压缩后会使缓存失效多少 token，并且延迟到 API 响应后才发出 boundary message，因为只有那时才能拿到真实的 cache\_deleted\_input\_tokens。

### 第三层：Context Collapse

这一层的设计最让我惊艳。

它不是替换消息，而是创建读时投影（read-time projection）。摘要存储在独立的 collapse store 中，原始消息不动。每次循环迭代入口，projectView() 回放 collapse 日志，生成一个压缩后的视图。

原始 messages 不变！这是 projection，不是 mutation。

为什么要这么设计？两个原因：跨 turn 持久化——collapse 记录存在 store 里，重启后依然有效；与 autocompact 解耦——如果 collapse 已经把 token 数降到阈值以下，autocompact 就不用跑了，省了一次 API 调用。

### 第四层：AutoCompact

最重量级。fork 一个独立的 API 调用来生成整个会话的摘要。内置熔断器——连续 3 次压缩失败后自动禁用。

错误恢复：三级瀑布

当 API 返回 `prompt_too_long` (413 错误) 时，不是直接报错，而是启动三级恢复：

第一级，Context Collapse drain——把所有待提交的 collapse 立即提交，释放空间，成功则重试。

第二级，Reactive Compact——紧急执行一次全量压缩，成功则重试。

第三级——全部失败，向用户报错。

类似地，`max_output_tokens` 也有两级恢复。先升级 token 上限（从 8k 到 64k 直接重试），再注入一条续写提示——大意是“你被截断了，从断点继续，不要道歉不要复述”——最多重试 3 次。

这条续写提示的措辞值得玩味。“no apology, no recap”——因为模型被截断后最常见的行为就是道歉 + 复述前文，白白浪费 token。一条精心调教的 meta prompt，解决了一个真实的工程问题。



rednote ID: 10819659153

## 8. 消息类型系统：被忽视的基础设施

Claude Code 定义了 7 种消息类型：  
AssistantMessage（模型响应）、  
UserMessage（用户输入/tool\_result）、  
SystemMessage（合成消息）、  
ProgressMessage（工具执行进度）、  
AttachmentMessage（动态上下文注入）、  
ToolUseSummaryMessage（工具调用摘要）、  
TombstoneMessage（废弃消息的墓碑标记）。

TombstoneMessage 值得单独说。当主模型过载、降级到 fallback 模型时，之前已经流式输出的 assistant messages 怎么办？它们包含了主模型的 thinking 签名，发给 fallback 模型会导致 400 错误。

Claude Code 的做法是 yield 一系列 tombstone 消息，告诉 UI 层“这些消息作废了，从界面上移除”。然后清空所有中间状态，用 fallback 模型重新开始。

这个设计体现了一个原则：流式系统的错误恢复，比批处理系统难一个数量级。你已经 yield 出去的东西，用户已经看到了，你不能假装它没发生过。只能

rednote ID: 10819659153

通过 tombstone 这种"追溯性撤回"机制来处理。

## 9. 不可变 API 消息：一个贯穿全局的设计哲学

这个细节贯穿了整个代码库：API 返回的消息对象，永远不被修改。

当需要在 yield 前修改消息时（比如给工具调用补充可观测字段），代码会先克隆再修改。而且只有真正添加了新字段才克隆，覆盖已有字段不克隆。

为什么这么执着于不可变？Prompt caching。

Anthropic 的 prompt caching 是基于字节匹配的。如果你修改了历史消息的任何一个字节，缓存就失效了。而 assistantMessages 数组里的消息最终会被塞回下一轮的 messages 发给 API。如果你修改了它们，cache hit rate 直接归零，每轮都要重新计算整个上下文。

对于一个长会话，这可能意味着每轮多花几万个 input token 的钱。一个不可变性约束，直接影响了 Claude Code 的运行成本。



## 10. 我的思考：为什么这个设计有效

读完 Claude Code 的源码，我总结了四个核心设计决策：

循环优于图

LangGraph 用图，CrewAI 用角色编排，AutoGen 用对话协议。Claude Code 用 `while(true)`。听起来原始，但这恰恰是最灵活的选择。图的节点和边是编译时确定的，循环的行为是运行时决定的。当你的 agent 需要根据中间结果动态调整策略时，图是约束，循环是自由。

递归优于编排

子 agent 就是递归调用主循环。新增的压缩策略、错误恢复机制、权限检查，自动对所有层级的 agent 生效。维护成本是  $O(1)$ ，不是  $O(n)$ 。

让模型做决策，框架做执行

Claude Code 不尝试在框架层面理解任务的依赖关系。它相信模型知道自己在干什么。框架只需要守住一个安全约束：有副作用的操作串行执行。其他的，

rednote ID: 10819659153

交给模型。

为真实世界设计，不为 demo 设计

四层压缩、三级错误恢复、max\_output\_tokens 续写、streaming fallback 的 tombstone 处理——这些东西在 demo 里完全用不到。一个 5 分钟的演示永远不会触发 autocompact，永远不会遇到 prompt\_too\_long，永远不会经历模型降级。但真实用户会。

这大概也是为什么 Claude Code 是目前最好用的 AI 编程工具——不是因为它的 prompt 写得多好，而是因为它的工程做得足够扎实。

写在最后：这篇文章只覆盖了 Claude Code 的核心 agent flow。还有很多值得深挖的系统没有展开，比如 MCP 集成、Hook 系统、Session Resume 机制、Memory 系统等。如果大家感兴趣，后续可以继续写。

如果你也在构建 AI agent 系统，我强烈建议你去读一下这份源码。不是为了抄，而是为了理解一个被真实用户大规模使用的 agent 系统，到底需要解决哪些问题。



rednote ID: 10819659153